

# PA1 — Stateful Tracking on the Web

**Project Release:** 09/24/2026

**Part 1 Deadline:** 10/1/2026

**Part 2 Deadline:** 10/13/2026

## Introduction

A new company, **Boogle**, is trying to compete in the online search game. The startup has everything—VC money, a cushy office, and free snacks—but there’s one problem: no one at the company knows anything about web development. **Boogle** has hired you as a web engineer to get their systems up and running.

In particular, they want to compete in the *stateful tracking* game—they’ve heard that their primary competitors get most of their mileage by tracking their users and selling them ads. Your task will be to get their infrastructure up and running and implement this stateful tracking, along with their main business function (an online search page).

## Goal

This project will introduce you to web programming and stateful, cookie-based tracking. You will implement a web server, deploy it to the public Internet using a service of your choice, implement a database, write some simple business logic for the **Boogle** page, and wire up the database to contain all the appropriate user tracking information.

**Note:** The project is split into four parts—this document contains just parts one and two. Parts three and four will build off of what you accomplish in one and two.

## Programming and Infrastructure

This project does not restrict you in terms of the software stack you choose to use. Indeed, web servers can be implemented in anything, from **assembly** to **JavaScript**. That being said, we recommend you explore building using **Flask**, a very simple Python-based web server framework. The API is straightforward and Python is a relatively easy programming language to learn and use.

On the infrastructure side, again, there are dozens of companies and providers that can host your web server. You can be as simple or complex as you wish, so long as you meet the spec requirements. For simplicity (and cost), we recommend you leverage an existing, hyperscalar cloud provider—e.g., **Google Cloud**, **Microsoft Azure**, or **Amazon AWS**. All three of these services have free tiers for deployment and hosting that should last you the duration of this project.

## Read this first!

This project asks you to develop technology that can legitimately infringe on people’s privacy. Attempting these same kinds of attacks against real people may result in fines, expulsion, and jail time. You must not use these techniques to attack or deanonymize anyone without authorization! You are required to respect the privacy and property rights of others at all times or you will fail the course.

## Forming a Group

This is a group project; you can work in a team of size at most two and one entry per team. You are not required to work with the same team on every PA. You and your team should collaborate closely on each part.

The code and other answers you submit must be entirely your team's own work. You may discuss the conceptualization of the project and the meaning of the questions with other teams, but you may not look at any part of someone else's solution or collaborate with anyone other than your team. You may consult published references, provided that you appropriately cite them (e.g., with program comments).

Solutions must be submitted to Gradescope.

## Index

- [Setup](#)
- [Part 1: A Simple Web Server](#)
- [Part 2: A Web Server that Tracks You](#)
- [Resources and Guidelines](#)

## Setup

We provide you with some starter code for parts one and two. You can download the starter code here: [https://webtracking.sysnet.ucsd.edu/pa1/pa1\\_starter.zip](https://webtracking.sysnet.ucsd.edu/pa1/pa1_starter.zip).

After unzipping the archive, you should see the following directory structure:

```
/
├── boogle.html
├── app.py
├── schema_handout.sql
├── dump_example.json
└── history_example.json
```

For this part of the assignment, you will primarily be writing in Python, should you choose to implement your web server via Flask. You may want to leverage a [Python virtual environment](#) to ensure you do not damage your system's global configuration. `boogle.html` and `app.py` are relevant for parts 1 and 2, whereas `schema_handout.sql`, `dump_example.json`, and `history_example.json` are relevant for only part 2.

## Part 1: A simple web server

In this part of the assignment, you will be implementing a very simple web server. The starter code in `app.py` instantiates an example web server written in Python Flask, which returns a simple “Hello World” HTML segment to the browser on the index route. Your goal will be to implement two routes: `/` and `/jsontest`, which we detail below.

### Implementing GET `/`

For the index route, your server must serve the `boogle.html` page we provide to you in the starter code. The file should be **unchanged**—we will test this explicitly in our code. The route should only receive `/GET` requests—all other request types should return an error. Make sure you return a 200 OK for valid requests.

### Implementing GET `/jsontest`

For the `/jsontest` route, your server must return a JSON blob of the following form:

```
{
  "data" : "I am in CSE190/CSE291!"
}
```

Make sure the content type you return is `application/json`, and return a 200 OK status code when receiving requests.

## Deploying your web server

After you build your simple web app, you have to *deploy* the application to the public Internet. This means finding a provider that can host your particular web server at an IP address that our infrastructure can access.

There are many options here—ranging from serverless products to fully fledged virtual machines—and many will work for your task. The only requirement is that we can reach your web server at an IP address and port that you control and is running your code. We recommend exploring options to run your own virtual machine via [Google Cloud](#), [Microsoft Azure](#), or [Amazon AWS](#). All three of these services have free tiers for deployment and hosting that should last you the duration of this project.

**If you have financial constraints that would render this part impossible, please reach out to staff immediately and we can help you.**

## Testing your server

We have set up automated infrastructure that can test your server implementation at <https://webtracking.sysnet.ucsd.edu/pa1/part1>. You simply put your IP address into the bar, and our autograder runs the test cases we will use to grade your submission and tells you which cases you passed or failed. Students are limited to seven submissions per day (per TSS ID), and so we recommend that you do not bombard the server with requests without properly testing and examining your code first.

## Submitting your work

You will submit a single file, called `server.txt`, to Gradescope as your final submission. The file will contain a single line—with the form `< ip.address >:< port >`. Note that your server **must be up** and running consistently when our grader runs (which will be some time after the project is due).

## Part 2: A web server that tracks you

You will now implement a more sophisticated web server that tracks users via a *first-party cookie*. You will also implement the searching functionality for Boogle, and store all the resultant information in a database to track user behavior.

### Implementing GET /

For every new, previously unseen request to your web server on the `/` route, your server will now set a cookie with the following requirements:

The cookie name must be `boogle_id`, and the corresponding value must match a UUID format (we recommend `uuid4`). The cookie must apply only to the `/` route, be a *persistent cookie* (e.g., set `Max-Age`), and use `Lax SameSite` preferences. Likely, since your server will not have HTTPS, you should not select the `Secure` cookie option. You can view the cookies being set by inspecting the network requests sent by your server on any major browser.

**Note: You should only set a cookie value if one was previously unset. Overwriting the cookie value will rendering the tracking useless.**

## Implementing GET /search

The `boogle.html` file we provide you makes a `GET` request to `/search` whenever someone presses the “search” button. Your server should receive the query (which is automatically populated in the `q` query parameter), store the query in a database (more on the database below), and return a `303 See Other` HTTP response code with the route `/`, which will trigger the browser to refresh the main page.

You should **not** accept empty searches (e.g., a query that has nothing but whitespace), and still redirect back to `/`.

## Implementing GET /api/history

The `boogle.html` page we provide also makes a `GET` request to the `/api/history` route every time it is loaded, which displays all the previous searches this specific client has made.

You should retrieve all the searches made by the client (in chronologically reverse order), and return them in the following JSON format:

```
{
  "client_id": "<UUID>",
  "searches": [
    { "query": "best tacos", "ts": "2026-09-16T18:04:11.512Z" },
    { "query": "cheap flights", "ts": "2026-09-16T18:03:02.104Z" }
  ]
}
```

Where “query” is the search query made, and “ts” is the time *when* the query was made. Take care to *only* return searches made by the specific client. If a new client is making a request (i.e., there is no cookie set on the browser), `searches` should return an empty array `[]`.

## Implementing GET /dump

The final route, `/dump`, dumps all the requests you have stored on clients that visit your website. It should follow the following JSON format:

```
{
  "clients": [
    {
      "client_id": "<UUID1>",
      "first_seen": "2026-09-16T18:02:58.001Z",
      "last_seen": "2026-09-16T18:04:11.512Z",
      "latest_ip": "137.110.222.15",
      "ips": ["137.110.222.15"],
      "searches": [{ "query": "best tacos", "ts": "2026-09-16T18:04:11.512Z" },
        { "query": "cheap flights", "ts": "2026-09-16T18:03:02.104Z" } ]
    },
    {
      "client_id": "<UUID2>",
      ...
    }
  ]
}
```

With one JSON entry in the `clients` array per client. Note that several fields, including `first_seen`, `last_seen`, and `latest_ip` require you to keep track of when you have seen the client and what IP address they are coming from, in addition to their search queries.

## Storing clients in a database

In order to enable this website functionality, you will need to keep track of user interactions with Boogle in a database. We **strongly recommend** you use [Postgres SQL](#), which is a state-of-the-art SQL database that is used by thousands of companies. That being said, you can use any database that enables you to produce the JSON specifications for `/api/history` and `/dump` above.

To incentivize you to use Postgres, however, we have included a nearly complete SQL schema file that you can directly input into a Postgres database in `schema_handout.sql`. But, if you still wish to use something else (like MongoDB, heaven forbid), note that the TAs and/or staff may not be able to help you.

## Testing your server

Similarly to part 1, we have set up automated infrastructure that can test your server implementation at <https://webtracking.sysnet.ucsd.edu/pa1/part2>. Same rate limits apply to this autograder, so be choosy with your submissions.

## Submitting your work

You will submit a single file, called `server.txt`, to Gradescope as your final submission. The file will contain a single line—with the form `< ip_address >:< port >`. Note that your server **must be up** and running consistently when our grader runs (which will be some time after the project is due).

## Resources and Guidelines

### Basic Resources

The web is a wild, wild place. Review the slides on web programming, and if you need more help, we recommend reviewing Mozilla Developer Network (MDN)'s page on [how the web works](#). It is a great primer on what we cover in class, and MDN is a great resource for all things web related.

### Cookies

[The MDN Docs on Cookies](#) is comprehensive and tells you a lot of what you might want to know about how they are implemented in practice by browsers.

### SQL

Outside of what we teach you in this class, [W3Schools has a simple SQL tutorial](#) that should be good enough for the queries in this project. If you need more advice on generating these queries, there are dozens of resources online.